

# Guide detaille - Ce qui est demande pour la Phase 5 Deploiement

---

## 1. But de la phase

La phase 5 demande de prouver que l'application Eco-Ressource peut etre deployee comme une vraie application professionnelle :

- conteneurisee avec Docker ;
- publiee sous forme d'images sur Docker Hub ;
- orchestree avec Kubernetes ;
- deployee sur une infrastructure accessible, idealement OpenStack ou des VMs ;
- automatisee avec Ansible ;
- surveillee avec des outils de monitoring ;
- presentee avec un scenario clair par tous les membres de l'equipe.

Le jury doit voir une application stable, accessible par URL, avec frontend, backend et base de donnees fonctionnels.

## 2. Architecture cible

L'architecture conseillee est :

```
Utilisateur
|
v
Frontend Angular - Nginx
|
v
Backend Spring Boot
|
v
Base de donnees MySQL
```

Dans Kubernetes :

```
Namespace eco-app
|
|-- frontend-deployment + frontend-service
|-- backend-deployment + backend-service
|-- mysql-deployment + mysql-service
|-- configmaps
|-- secrets
|-- hpa
|-- monitoring
```

## 3. Fichiers a preparer

### 3.1. Dossier Docker

Chemin :

```
deployment/docker/
```

Fichiers attendus :

```
deployment/docker/frontend.Dockerfile
deployment/docker/backend.Dockerfile
deployment/docker/docker-compose.yml
deployment/docker/nginx.conf
deployment/docker/COMMANDES_DOCKER.md
```

Role des fichiers :

- **frontend.Dockerfile** : construire l'image du frontend Angular avec un build multi-stage.
- **backend.Dockerfile** : construire l'image du backend Spring Boot avec un build multi-stage.
- **docker-compose.yml** : tester localement frontend + backend + base de donnees.
- **nginx.conf** : configurer Nginx pour servir Angular.
- **COMMANDES\_DOCKER.md** : documenter les commandes Docker utiles.

### 3.2. Dossier Kubernetes

Chemin a creer :

```
deployment/k8s/
```

Fichiers a creer :

```
deployment/k8s/namespace.yaml
deployment/k8s/mysql-secret.yaml
deployment/k8s/mysql-configmap.yaml
deployment/k8s/mysql-deployment.yaml
deployment/k8s/mysql-service.yaml
deployment/k8s/backend-secret.yaml
deployment/k8s/backend-configmap.yaml
deployment/k8s/backend-deployment.yaml
deployment/k8s/backend-service.yaml
deployment/k8s/frontend-configmap.yaml
deployment/k8s/frontend-deployment.yaml
deployment/k8s/frontend-service.yaml
deployment/k8s/hpa-backend.yaml
```

```
deployment/k8s/hpa-frontend.yaml  
deployment/k8s/ingress.yaml
```

Role des fichiers :

- `namespace.yaml` : creer un espace logique `eco-app`.
- `mysql-secret.yaml` : stocker le mot de passe MySQL.
- `mysql-configmap.yaml` : stocker les configurations non sensibles de MySQL.
- `mysql-deployment.yaml` : lancer le pod MySQL.
- `mysql-service.yaml` : exposer MySQL dans le cluster.
- `backend-secret.yaml` : stocker les informations sensibles du backend.
- `backend-configmap.yaml` : stocker les variables non sensibles du backend.
- `backend-deployment.yaml` : lancer les pods backend.
- `backend-service.yaml` : exposer le backend dans le cluster.
- `frontend-configmap.yaml` : stocker la configuration frontend si necessaire.
- `frontend-deployment.yaml` : lancer les pods frontend.
- `frontend-service.yaml` : exposer le frontend.
- `hpa-backend.yaml` : autoscaling du backend.
- `hpa-frontend.yaml` : autoscaling du frontend si necessaire.
- `ingress.yaml` : fournir une URL propre, si Ingress est disponible.

### 3.3. Dossier Ansible

Chemin a creer :

```
deployment/ansible/
```

Fichiers a creer :

```
deployment/ansible/inventory.ini  
deployment/ansible/group_vars/all.yml  
deployment/ansible/playbooks/01-prerequisites.yml  
deployment/ansible/playbooks/02-install-kubernetes.yml  
deployment/ansible/playbooks/03-init-master.yml  
deployment/ansible/playbooks/04-join-workers.yml  
deployment/ansible/playbooks/05-deploy-app.yml  
deployment/ansible/playbooks/06-install-monitoring.yml
```

Role des fichiers :

- `inventory.ini` : liste des VMs master et worker.
- `all.yml` : variables globales.
- `01-prerequisites.yml` : installation des prerequis systeme.
- `02-install-kubernetes.yml` : installation de kubeadm, kubelet, kubectl.
- `03-init-master.yml` : initialisation du master Kubernetes.

- `04-join-workers.yml` : ajout des workers au cluster.
- `05-deploy-app.yml` : deploiement des fichiers Kubernetes.
- `06-install-monitoring.yml` : installation du monitoring.

### 3.4. Dossier Monitoring

Chemin a creer :

```
deployment/monitoring/
```

Fichiers a creer :

```
deployment/monitoring/prometheus-values.yaml  
deployment/monitoring/grafana-dashboard.json  
deployment/monitoring/alerts.yaml  
deployment/monitoring/README_MONITORING.md
```

Role des fichiers :

- `prometheus-values.yaml` : configuration Prometheus/Grafana avec Helm.
- `grafana-dashboard.json` : dashboard Grafana exporte.
- `alerts.yaml` : regles d'alertes.
- `README_MONITORING.md` : explication du monitoring.

## 4. Etape 1 - Tester l'application localement

Avant Docker et Kubernetes, il faut verifier que l'application fonctionne normalement.

Frontend :

```
npm install  
npm run build -- --configuration production  
npm start
```

Backend :

```
mvn clean package -DskipTests  
mvn spring-boot:run
```

Base de donnees :

```
mysql -u root -p
```

```
CREATE DATABASE eco_ressource_db;
```

Preuves a garder :

- capture du frontend ;
- capture d'un appel backend ;
- capture de la base de donnees ;
- logs sans erreur.

## 5. Etape 2 - Construire les images Docker

### 5.1. Construire l'image frontend

Depuis le dossier racine du frontend :

```
docker build -f deployment/docker/frontend.Dockerfile -t eco-frontend:v1 .
```

Verifier :

```
docker images
```

Tester :

```
docker run --rm -p 8080:80 eco-frontend:v1
```

Ouvrir :

```
http://localhost:8080
```

### 5.2. Construire l'image backend

Depuis le dossier racine du frontend, si le backend est dans `../eco-ressource-backend-main` :

```
docker build -f deployment/docker/backend.Dockerfile -t eco-backend:v1 ../eco-ressource-backend-main
```

Verifier :

```
docker images
```

Tester avec une base externe :

```
docker run --rm -p 9090:9090 \  
-e \  
SPRING_DATASOURCE_URL="jdbc:mysql://host.docker.internal:3306/eco_ressource_db? \  
useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true&createDatabaseIfNotEx \  
ist=true" \  
-e SPRING_DATASOURCE_USERNAME=root \  
-e SPRING_DATASOURCE_PASSWORD=root \  
eco-backend:v1
```

Verifier :

```
curl http://localhost:9090/actuator/health
```

## 6. Etape 3 - Tester avec Docker Compose

Commande :

```
docker compose -f deployment/docker/docker-compose.yml up --build
```

Dans un autre terminal :

```
docker ps  
docker logs eco-db  
docker logs eco-backend  
docker logs eco-frontend
```

Tester :

```
Frontend : http://localhost:8080  
Backend  : http://localhost:9090
```

Arreter :

```
docker compose -f deployment/docker/docker-compose.yml down
```

Nettoyer avec volumes si necessaire :

```
docker compose -f deployment/docker/docker-compose.yml down -v
```

Ce que le jury peut demander :

- pourquoi trois conteneurs ?
- comment le backend trouve la base de données ?
- pourquoi utiliser un réseau Docker ?
- pourquoi utiliser un build multi-stage ?

## 7. Etape 4 - Pousser les images sur Docker Hub

Remplacer `<dockerhub_user>` par le compte Docker Hub de l'équipe.

Connexion :

```
docker login
```

Tag frontend :

```
docker tag eco-frontend:v1 <dockerhub_user>/eco-frontend:v1
```

Tag backend :

```
docker tag eco-backend:v1 <dockerhub_user>/eco-backend:v1
```

Push :

```
docker push <dockerhub_user>/eco-frontend:v1  
docker push <dockerhub_user>/eco-backend:v1
```

Vérifier :

```
docker pull <dockerhub_user>/eco-frontend:v1  
docker pull <dockerhub_user>/eco-backend:v1
```

Preuves :

- sortie `docker push` ;
- page Docker Hub ;
- noms exacts des images.

## 8. Etape 5 - Preparer le cluster Kubernetes

### 8.1. Option OpenStack ou VMs

Creer au minimum :

```
1 VM master
1 VM worker
```

Ideal :

```
1 VM master
2 VM workers
```

Exemple :

```
master   : 2 CPU, 4 Go RAM, Ubuntu Server
worker1  : 2 CPU, 4 Go RAM, Ubuntu Server
worker2  : 2 CPU, 4 Go RAM, Ubuntu Server
```

Ports a verifier :

- SSH : 22
- Kubernetes API : 6443
- kubelet : 10250
- NodePort : 30000-32767
- HTTP : 80
- HTTPS : 443

### 8.2. Verifier les nodes

Sur la machine de controle :

```
kubectl get nodes
```

Resultat attendu :

| NAME    | STATUS | ROLES         | AGE | VERSION |
|---------|--------|---------------|-----|---------|
| master  | Ready  | control-plane | ... |         |
| worker1 | Ready  | <none>        | ... |         |
| worker2 | Ready  | <none>        | ... |         |



## 9. Etape 6 - Creer les manifests Kubernetes

### 9.1. namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: eco-app
```

### 9.2. mysql-secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: mysql-secret
  namespace: eco-app
type: Opaque
stringData:
  MYSQL_ROOT_PASSWORD: root
```

### 9.3. mysql-configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: mysql-config
  namespace: eco-app
data:
  MYSQL_DATABASE: eco_ressource_db
```

### 9.4. mysql-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: eco-db
  namespace: eco-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: eco-db
  template:
    metadata:
      labels:
```

```
  app: eco-db
spec:
  containers:
    - name: mysql
      image: mysql:8.4
      ports:
        - containerPort: 3306
      envFrom:
        - configMapRef:
            name: mysql-config
        - secretRef:
            name: mysql-secret
      resources:
        requests:
          cpu: "250m"
          memory: "512Mi"
        limits:
          cpu: "500m"
          memory: "1Gi"
```

## 9.5. mysql-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: eco-db
  namespace: eco-app
spec:
  selector:
    app: eco-db
  ports:
    - port: 3306
      targetPort: 3306
  type: ClusterIP
```

## 9.6. backend-secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: backend-secret
  namespace: eco-app
type: Opaque
stringData:
  SPRING_DATASOURCE_USERNAME: root
  SPRING_DATASOURCE_PASSWORD: root
```

## 9.7. backend-configmap.yaml

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-config
  namespace: eco-app
data:
  SERVER_PORT: "9090"
  SPRING_DATASOURCE_URL: "jdbc:mysql://eco-db:3306/eco_ressource_db?
useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true&createDatabaseIfNotEx
ist=true"
  FILE_UPLOAD_DIR: "/app/uploads"

```

## 9.8. backend-deployment.yaml

Remplacer `<dockerhub_user>` :

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: eco-backend
  namespace: eco-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: eco-backend
  template:
    metadata:
      labels:
        app: eco-backend
    spec:
      containers:
        - name: eco-backend
          image: <dockerhub_user>/eco-backend:v1
          imagePullPolicy: Always
          ports:
            - containerPort: 9090
          envFrom:
            - configMapRef:
                name: backend-config
            - secretRef:
                name: backend-secret
          readinessProbe:
            httpGet:
              path: /actuator/health
              port: 9090
            initialDelaySeconds: 30
            periodSeconds: 10
          livenessProbe:
            httpGet:

```

```
    path: /actuator/health
    port: 9090
    initialDelaySeconds: 60
    periodSeconds: 20
  resources:
    requests:
      cpu: "250m"
      memory: "512Mi"
    limits:
      cpu: "1"
      memory: "1Gi"
```

## 9.9. backend-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: eco-backend
  namespace: eco-app
spec:
  selector:
    app: eco-backend
  ports:
    - port: 9090
      targetPort: 9090
  type: ClusterIP
```

## 9.10. frontend-deployment.yaml

Remplacer `<dockerhub_user>` :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: eco-frontend
  namespace: eco-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: eco-frontend
  template:
    metadata:
      labels:
        app: eco-frontend
    spec:
      containers:
        - name: eco-frontend
          image: <dockerhub_user>/eco-frontend:v1
```

```

    imagePullPolicy: Always
    ports:
      - containerPort: 80
    resources:
      requests:
        cpu: "100m"
        memory: "128Mi"
      limits:
        cpu: "500m"
        memory: "256Mi"

```

### 9.11. frontend-service.yaml

Pour une demo simple :

```

apiVersion: v1
kind: Service
metadata:
  name: eco-frontend
  namespace: eco-app
spec:
  selector:
    app: eco-frontend
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
  type: NodePort

```

Acces :

```
http://<IP_WORKER_OU_MASTER>:30080
```

### 9.12. hpa-backend.yaml

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: eco-backend-hpa
  namespace: eco-app
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: eco-backend
  minReplicas: 2
  maxReplicas: 5

```

```
metrics:
  - type: Resource
    resource:
      name: cpu
    target:
      type: Utilization
      averageUtilization: 70
```

## 10. Etape 7 - Deployer sur Kubernetes

Appliquer les fichiers :

```
kubectl apply -f deployment/k8s/namespace.yaml
kubectl apply -f deployment/k8s/mysql-secret.yaml
kubectl apply -f deployment/k8s/mysql-configmap.yaml
kubectl apply -f deployment/k8s/mysql-deployment.yaml
kubectl apply -f deployment/k8s/mysql-service.yaml
kubectl apply -f deployment/k8s/backend-secret.yaml
kubectl apply -f deployment/k8s/backend-configmap.yaml
kubectl apply -f deployment/k8s/backend-deployment.yaml
kubectl apply -f deployment/k8s/backend-service.yaml
kubectl apply -f deployment/k8s/frontend-deployment.yaml
kubectl apply -f deployment/k8s/frontend-service.yaml
kubectl apply -f deployment/k8s/hpa-backend.yaml
```

Ou appliquer tout le dossier :

```
kubectl apply -f deployment/k8s/
```

Vérifier :

```
kubectl get all -n eco-app
kubectl get pods -n eco-app -o wide
kubectl get svc -n eco-app
kubectl get configmap -n eco-app
kubectl get secret -n eco-app
kubectl get hpa -n eco-app
```

Voir les logs :

```
kubectl logs deployment/eco-backend -n eco-app
kubectl logs deployment/eco-frontend -n eco-app
```

Diagnostiquer un problème :

```
kubectl describe pod <pod-name> -n eco-app  
kubectl get events -n eco-app --sort-by=.metadata.creationTimestamp
```

Redemarrer un deploiement :

```
kubectl rollout restart deployment/eco-backend -n eco-app  
kubectl rollout status deployment/eco-backend -n eco-app
```

Supprimer l'application :

```
kubectl delete -f deployment/k8s/
```

## 11. Etape 8 - Installer metrics-server pour HPA

Verifier si metrics-server existe :

```
kubectl get deployment metrics-server -n kube-system
```

Installer :

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-  
server/releases/latest/download/components.yaml
```

Verifier :

```
kubectl top nodes  
kubectl top pods -n eco-app  
kubectl get hpa -n eco-app
```

Si `kubectl top` ne marche pas, le HPA ne pourra pas calculer les metriques CPU/RAM.

## 12. Etape 9 - Tester la resilience

Lister les pods :

```
kubectl get pods -n eco-app
```

Supprimer un pod backend :

```
kubectl delete pod <backend-pod-name> -n eco-app
```

Observer la recreation :

```
kubectl get pods -n eco-app -w
```

Ce qu'il faut expliquer :

- Kubernetes compare l'etat actuel avec l'etat desire.
- Le Deployment demande 2 replicas backend.
- Si un pod tombe, Kubernetes le remplace automatiquement.

## 13. Etape 10 - Tester l'autoscaling

Lancer une charge sur le backend :

```
kubectl run -i --tty load-generator --rm --image=busybox:1.36 --restart=Never -- /bin/sh
```

Dans le shell du pod :

```
while true; do wget -q -O- http://eco-backend:9090/actuator/health; done
```

Dans un autre terminal :

```
kubectl get hpa -n eco-app -w  
kubectl get pods -n eco-app -w
```

Resultat attendu :

- la charge CPU augmente ;
- le HPA augmente les replicas ;
- apres l'arret de la charge, le nombre de replicas redescend.

## 14. Etape 11 - Installer monitoring Prometheus et Grafana

Installer Helm si necessaire.

Ajouter le repo :



```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

Installer kube-prometheus-stack :

```
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --create-namespace
```

Verifier :

```
kubectl get pods -n monitoring
kubectl get svc -n monitoring
```

Acceder a Grafana en port-forward :

```
kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring
```

Ouvrir :

```
http://localhost:3000
```

Recuperer le mot de passe Grafana :

```
kubectl get secret monitoring-grafana -n monitoring -o jsonpath="{.data.admin-password}" | base64 -d
```

Dashboard a montrer :

- CPU pods ;
- RAM pods ;
- nombre de replicas ;
- etat des nodes ;
- erreurs applicatives si disponibles ;
- disponibilite du backend.

## 15. Etape 12 - Ajouter un scenario IA de monitoring intelligent

Le scenario IA peut etre simple mais clair.

Exemples acceptables :

- detection d'anomalie CPU/RAM ;
- detection d'un taux d'erreur inhabituel ;
- detection de temps de reponse trop eleves ;
- recommandation d'augmenter les replicas ;
- recommandation d'augmenter CPU/RAM ;
- prediction d'une charge elevee.

Exemple de logique :

```
Si CPU backend > 80% pendant 5 minutes :  
  anomalie detectee  
  recommandation : augmenter maxReplicas HPA de 5 a 8  
  recommandation : augmenter CPU limit de 1 a 1.5
```

Preuves a montrer :

- capture Grafana ;
- alerte Prometheus ;
- tableau de recommandations ;
- logs analyses ;
- explication orale.

## 16. Etape 13 - Automatiser avec Ansible

### 16.1. inventory.ini

```
[masters]  
master ansible_host=192.168.1.10 ansible_user=ubuntu  
  
[workers]  
worker1 ansible_host=192.168.1.11 ansible_user=ubuntu  
worker2 ansible_host=192.168.1.12 ansible_user=ubuntu  
  
[k8s:children]  
masters  
workers
```

### 16.2. Tester la connexion

```
ansible -i deployment/ansible/inventory.ini all -m ping
```

### 16.3. Playbook prerequis

Objectif :

- mettre à jour les paquets ;
- désactiver swap ;
- installer curl, apt-transport-https, ca-certificates ;
- configurer les modules kernel ;
- installer containerd.

Commande :

```
ansible-playbook -i deployment/ansible/inventory.ini  
deployment/ansible/playbooks/01-prerequisites.yml
```

## 16.4. Playbook installation Kubernetes

Objectif :

- installer kubeadm ;
- installer kubelet ;
- installer kubectl ;
- activer kubelet.

Commande :

```
ansible-playbook -i deployment/ansible/inventory.ini  
deployment/ansible/playbooks/02-install-kubernetes.yml
```

## 16.5. Playbook init master

Objectif :

- lancer `kubeadm init` ;
- configurer kubeconfig ;
- installer Calico ou Flannel ;
- générer la commande `kubeadm join`.

Commande :

```
ansible-playbook -i deployment/ansible/inventory.ini  
deployment/ansible/playbooks/03-init-master.yml
```

## 16.6. Playbook join workers

Objectif :

- connecter les workers au cluster.

Commande :

```
ansible-playbook -i deployment/ansible/inventory.ini  
deployment/ansible/playbooks/04-join-workers.yml
```

## 16.7. Playbook deploy app

Objectif :

- copier les manifests Kubernetes ;
- appliquer les fichiers YAML ;
- verifier les pods.

Commande :

```
ansible-playbook -i deployment/ansible/inventory.ini  
deployment/ansible/playbooks/05-deploy-app.yml
```

## 17. Etape 14 - Scenario de validation devant le jury

Ordre conseille :

1. Presenter rapidement l'application.
2. Montrer l'architecture globale.
3. Montrer les Dockerfiles.
4. Construire ou montrer les images Docker.
5. Montrer Docker Hub.
6. Montrer les fichiers Kubernetes.
7. Lancer `kubectl get all -n eco-app`.
8. Ouvrir l'application via l'URL.
9. Tester une fonctionnalite metier.
10. Montrer logs et monitoring.
11. Supprimer un pod et montrer la recreation.
12. Montrer HPA et autoscaling.
13. Montrer Ansible et expliquer les playbooks.
14. Chaque membre explique sa partie.

Commandes de demo rapides :

```
docker images  
docker ps  
kubectl get nodes  
kubectl get all -n eco-app  
kubectl get hpa -n eco-app  
kubectl top pods -n eco-app  
kubectl logs deployment/eco-backend -n eco-app
```

```
kubectl delete pod <pod-name> -n eco-app  
kubectl get pods -n eco-app -w
```

## 18. Questions probables du jury et reponses

Question : pourquoi Kubernetes ?

Reponse :

Kubernetes permet de gerer automatiquement les pods, les services, les replicas, le redemarrage automatique, la configuration, les secrets et l'autoscaling.

Question : pourquoi Ansible ?

Reponse :

Ansible automatise l'installation et la configuration du cluster. Cela rend le deployment reproductible et evite les erreurs manuelles.

Question : difference entre ConfigMap et Secret ?

Reponse :

ConfigMap stocke les configurations non sensibles. Secret stocke les informations sensibles comme les mots de passe.

Question : pourquoi Docker Hub ?

Reponse :

Kubernetes doit pouvoir recuperer les images depuis un registre. Docker Hub permet de stocker et distribuer les images frontend et backend.

Question : comment diagnostiquer un pod qui crash ?

Reponse :

```
kubectl get pods -n eco-app  
kubectl describe pod <pod-name> -n eco-app  
kubectl logs <pod-name> -n eco-app  
kubectl get events -n eco-app
```

Question : comment prouver la haute disponibilite ?

Reponse :

On supprime un pod manuellement. Le Deployment Kubernetes recree automatiquement un nouveau pod pour respecter le nombre de replicas demande.

## 19. Checklist finale pour obtenir la note complete

### Docker

- Dockerfile frontend existe.
- Dockerfile backend existe.
- Les deux Dockerfiles sont multi-stage.
- La base de donnees tourne dans un conteneur separe.
- Docker Compose lance frontend, backend et DB.
- Les services communiquent sans erreur.
- Les images sont poussees sur Docker Hub.

### Kubernetes

- Namespace cree.
- Deployments crees.
- Services crees.
- ConfigMaps crees.
- Secrets crees.
- Pods en **Running**.
- Pas de **CrashLoopBackOff**.
- Application accessible depuis l'exterieur.
- HPA configure.
- Requests/limits configures.

### Infrastructure

- Cluster stable sur OpenStack ou VMs.
- Nodes en **Ready**.
- Acces SSH fonctionnel.
- URL de l'application disponible.

### Monitoring

- Prometheus installe.
- Grafana installe.
- Dashboard disponible.
- Metriques CPU/RAM visibles.
- Logs consultables.
- Alerte ou scenario IA prepare.

## Ansible

- Inventory pret.
- Playbooks prets.
- Connexion Ansible testee.
- Installation cluster expliquée.
- Deploiement applicatif automatisable.

## Presentation

- Architecture claire.
- Scenario de validation prepare.
- Chaque membre parle.
- Chaque membre connait sa partie.
- Les commandes importantes sont pretes.
- Des captures sont disponibles en backup.

## 20. Commandes resumees a garder dans un fichier de demo

```
# Docker
docker compose -f deployment/docker/docker-compose.yml up --build
docker ps
docker images
docker logs eco-backend

# Docker Hub
docker login
docker tag eco-frontend:v1 <dockerhub_user>/eco-frontend:v1
docker tag eco-backend:v1 <dockerhub_user>/eco-backend:v1
docker push <dockerhub_user>/eco-frontend:v1
docker push <dockerhub_user>/eco-backend:v1

# Kubernetes
kubectl apply -f deployment/k8s/
kubectl get nodes
kubectl get all -n eco-app
kubectl get pods -n eco-app -o wide
kubectl get svc -n eco-app
kubectl get hpa -n eco-app
kubectl top pods -n eco-app
kubectl logs deployment/eco-backend -n eco-app

# Resilience
kubectl delete pod <pod-name> -n eco-app
kubectl get pods -n eco-app -w

# Monitoring
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install monitoring prometheus-community/kube-prometheus-stack --namespace
```

```
monitoring --create-namespace
kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring

# Ansible
ansible -i deployment/ansible/inventory.ini all -m ping
ansible-playbook -i deployment/ansible/inventory.ini
deployment/ansible/playbooks/01-prerequisites.yml
ansible-playbook -i deployment/ansible/inventory.ini
deployment/ansible/playbooks/02-install-kubernetes.yml
ansible-playbook -i deployment/ansible/inventory.ini
deployment/ansible/playbooks/03-init-master.yml
ansible-playbook -i deployment/ansible/inventory.ini
deployment/ansible/playbooks/04-join-workers.yml
ansible-playbook -i deployment/ansible/inventory.ini
deployment/ansible/playbooks/05-deploy-app.yml
```

## 21. Livrable final attendu

A la fin, le dossier **deployment/** doit permettre a quelqu'un de comprendre et reproduire le deploiement.

Livrables minimum :

- fichiers Docker ;
- fichiers Kubernetes ;
- fichiers Ansible ;
- fichier monitoring ;
- guide de commandes ;
- scenario de validation ;
- presentation Kubernetes et Ansible ;
- captures ou preuves de fonctionnement.

Objectif final :

Une application Eco-Ressource accessible, stable, conteneurisee, deployee sur Kubernetes, automatisee avec Ansible et surveillee avec monitoring.